

Building PhysSplat: A Step-by-Step Tutorial

From an empty folder to a photo-to-interactive-physics demo in the browser

Mohamad Salman

August 2026

Abstract

This is the companion tutorial to the PhysSplat design document. Where the design document says *what* the system is, this document walks through *building* it: 70 numbered steps organized into 9 phases, each with an explanation of what you are doing and why, code or commands where they help, and a checkpoint that tells you whether the step worked before you move on. Part I teaches all the background knowledge the build assumes: rigid-body physics, rotations, 3D Gaussian Splatting, graph neural networks, learned simulation, and the tool stack. Part II is the build itself. Read Part I once, then keep it open as a reference while working through Part II.

Contents

How to Use This Document	2
I Background Knowledge	2
1 Vectors, Rotations, and Poses	2
2 Rigid-Body Physics in Ten Minutes	3
3 3D Gaussian Splatting	4
4 Neural Networks and Graph Neural Networks	4
5 Learned Simulation: The GNS Recipe	5
6 The Tool Stack	6
II The Build	8
7 Phase 0: Environment and Skeleton	8
Step 1: Install the toolchain	8
Step 2: Create the project and Python environment	8
Step 3: Verify the Apple GPU works	8
Step 4: Lay out the repository	8
Step 5: First Rerun hello-world	8

8	Phase 1: The Synthetic Dataset	9
	Step 6: Define the object family	9
	Step 7: Build one static scene in PyBullet	9
	Step 8: Record trajectories	9
	Step 9: Add recorded forces (the poke-and-grab channel)	9
	Step 10: Write the shared particle sampler	10
	Step 11: Write the dataset to HDF5	10
	Step 12: Visual audit in Rerun	10
	Step 13: Generate the full dataset	11
	Step 14: Compute normalization statistics	11
	Step 15: Build the PyTorch <code>Dataset</code>	11
9	Phase 2: The Graph Network Simulator	11
	Step 16: The graph builder	11
	Step 17: Unit-test the graph builder	11
	Step 18: Encoder and processor	12
	Step 19: The per-body rigid decoder	12
	Step 20: The fallback per-particle decoder plus Kabsch projection	12
	Step 21: The integrator	12
	Step 22: Assemble the one-step model and overfit a single trajectory	13
	Step 23: The rollout function	13
10	Phase 3: Training	13
	Step 24: The training loop	13
	Step 25: Noise injection	13
	Step 26: Short training run on MPS	13
	Step 27: Rollout-based validation	13
	Step 28: Full training run	14
	Step 29: Qualitative milestones, in order	14
	Step 30: Rollout fine-tuning	14
	Step 31: Head comparison (only if needed)	14
11	Phase 4: Evaluation	14
	Step 32: Metrics harness	14
	Step 33: Side-by-side videos	14
	Step 34: Stress tests	14
	Step 35: Numbers table	14
	Step 36: Failure gallery	15
12	Phase 5: Local Interactive Demo	15
	Step 37: The scene packet format	15
	Step 38: WebSocket physics server	15
	Step 39: Minimal Three.js client	15
	Step 40: Mouse-to-force: pokes and grabs	15
	Step 41: Reset and settle behavior	16
	Step 42: Playtest	16
13	Phase 6: The Reconstruction Front End	16

Step 43: Capture test photos	16
Step 44: Background removal	16
Step 45: Run single-image reconstruction	16
Step 46: Gravity alignment and metric scale	16
Step 47: Ground plane via RANSAC	16
Step 48: Cluster into objects	17
Step 49: Per-body physical properties	17
Step 50: Resample physics particles	17
Step 51: Emit a scene packet	17
Step 52: End-to-end local test	17
14 Phase 7: Browser Inference and Splat Rendering	17
Step 53: Export the model to ONNX	17
Step 54: Parity test, Python vs. ONNX Runtime	17
Step 55: Scaffold the web app	18
Step 56: TypeScript graph builder and integrator	18
Step 57: The physics loop in the browser	18
Step 58: Splat rendering with per-body transforms	18
Step 59: Port interaction	18
Step 60: The scene endpoint, or static packets	18
Step 61: Deploy	19
Step 62: Cross-device pass	19
15 Phase 8: Polish and Writeup	19
Step 63: UI polish	19
Step 64: The demo video	19
Step 65: README and writeup	19
Glossary	19

How to Use This Document

The build has 9 phases. Each phase ends with something you can see or run. Do them in order; later phases import code from earlier ones.

Phase	What you build	Steps	Rough time
0	Environment and repository skeleton	1–5	half a day
1	Synthetic physics dataset (PyBullet)	6–16	1 week
2	The graph network simulator	17–28	1–2 weeks
3	Training it	29–36	1 week (+ compute)
4	Evaluating it	37–41	2–3 days
5	Local interactive demo	42–47	3–4 days
6	Photo $\rightarrow$ 3D reconstruction pipeline	48–57	1 week
7	Browser inference and splat rendering	58–67	1–2 weeks
8	Polish, deploy, write up	68–70	3–4 days

Box legend. Blue *Concept* boxes teach an idea the first time it is needed. Violet *Why this matters* boxes connect a step to the big picture. Green *Checkpoint* boxes are pass/fail tests; do not continue past a failing checkpoint. Orange *Common trap* boxes are mistakes that cost days if you hit them blind.

The one-paragraph map of what you are building. A photo of simple objects (blocks, pencils, erasers) on a table is lifted into a cloud of 3D Gaussians (a modern photorealistic 3D format). That cloud is split into individual objects automatically. Each object is dotted with a few hundred surface points called *physics particles*. A graph neural network, trained beforehand on millions of examples of simulated physics, repeatedly predicts how those particles move under gravity, contact, and your mouse pokes. The Gaussians ride along rigidly with their object, so what you see in the browser is the photo-derived scene obeying learned physics at 60 frames per second.

Part I

Background Knowledge

Everything in this part is used later. If a section is already familiar, skim its boxed summaries and move on.

1 Vectors, Rotations, and Poses

A 3D **vector** $\mathbf{x} = (x, y, z)$ describes a position or direction. We use z as “up” throughout, so gravity is $\mathbf{g} = (0, 0, -9.81) \text{ m/s}^2$.

A **rigid body** is an object that moves without changing shape. Its configuration at any instant, called its **pose**, is fully described by two things: where it is (a translation vector $\mathbf{t} \in \mathbb{R}^3$) and how it is turned (a rotation).

Concept: Three ways to write a rotation

- **Rotation matrix $\mathbf{R}$:** a 3×3 matrix with $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$ and $\det \mathbf{R} = 1$. Rotating a point is matrix multiplication: $\mathbf{x}' = \mathbf{R}\mathbf{x}$. Easy to compose and apply, 9 numbers.
- **Quaternion $\mathbf{q} = (w, x, y, z)$,** unit length: 4 numbers, no gimbal problems, the storage format used by PyBullet, Gaussian splat files, and Three.js. You will convert quaternions to matrices often; every library has a helper.
- **Axis-angle / rotation vector $\boldsymbol{\omega}\Delta t$:** a vector whose direction is the rotation axis and length is the angle. This is how angular velocity turns into an incremental rotation: $\mathbf{R}_{t+1} = \exp([\boldsymbol{\omega}\Delta t]_\times) \mathbf{R}_t$, where $[\cdot]_\times$ is the skew-symmetric cross-product matrix and $\exp$ is the matrix exponential (Rodrigues' formula computes it in closed form; `scipy.spatial.transform.Rotation.from_rotvec` does it for you).

The set of all poses $(\mathbf{R}, \mathbf{t})$ is called **SE(3)**. When this tutorial says “integrate on SE(3),” it just means: update the translation with the linear velocity and update the rotation with the angular velocity, using the formulas above.

Body frame vs. world frame. Each body carries its own coordinate system centered on it. A point that sits at body-frame offset $\mathbf{r}$ appears in the world at $\mathbf{x} = \mathbf{R}\mathbf{r} + \mathbf{t}$. This one formula powers the whole project: if we know each particle's fixed body-frame offset and the body's current pose, we know every particle's world position, and vice versa.

2 Rigid-Body Physics in Ten Minutes

State. A rigid body's motion state is four quantities: position $\mathbf{t}$, rotation $\mathbf{R}$, linear velocity $\mathbf{v}$ (m/s), and angular velocity $\boldsymbol{\omega}$ (rad/s, pointing along the spin axis).

Newton, twice. Forces change linear velocity ($\mathbf{F} = m\mathbf{a}$); torques change angular velocity ($\boldsymbol{\tau} = \mathbf{I}\boldsymbol{\alpha}$, where $\mathbf{I}$ is the **inertia tensor**, the rotational analog of mass). A pencil has tiny inertia about its long axis and large inertia about the others, which is why it spins easily lengthwise and why we feed the inertia diagonal to the neural network as a feature.

Integration. A simulator advances time in small steps Δt . The scheme used everywhere in this project is **semi-implicit Euler**: update velocity first, then use the *new* velocity to update position:

$$\mathbf{v}_{t+1} = \mathbf{v}_t + \mathbf{a} \Delta t, \quad \mathbf{t}_{t+1} = \mathbf{t}_t + \mathbf{v}_{t+1} \Delta t, \quad (1)$$

and the same pattern for rotation. It is one line more than naive Euler and dramatically more stable for oscillating and contacting systems.

Concept: Contact, friction, restitution

When bodies touch, the simulator must prevent interpenetration and apply friction:

- **Normal contact force** pushes bodies apart along the contact normal. Classical engines solve constraint equations; our neural network will *learn* to produce the same effect from data.
- **Friction coefficient μ** (typically 0.3–0.9 for everyday objects) scales how much tangential force resists sliding. Higher μ : things grip. Lower: things slide.
- **Restitution $e \in [0, 1]$** is bounciness: 0 is a dead thud, 1 is a perfect bounce. Desk

objects are 0–0.4.

These are exactly the parameters we randomize in training data so the learned model covers the whole plausible range.

Why learned physics at all? Classical engines need each object as a clean mesh with known parameters. Our objects come from a photo: fuzzy, incomplete, unlabeled. A learned simulator that operates on *surface point clouds* accepts exactly what reconstruction produces. The learned model imitates a classical engine (PyBullet generates its training data) but consumes a photo-compatible representation.

3 3D Gaussian Splatting

Concept: What a Gaussian splat is

A scene is represented as a cloud of tens of thousands of small colored, semi-transparent 3D blobs. Each blob (a “Gaussian”) has:

- a **mean** $\mu \in \mathbb{R}^3$: its center position;
- a **covariance** Σ : its shape and orientation, stored as 3 scales plus a quaternion (an ellipsoid);
- an **opacity** α ; and
- **color**, stored as spherical-harmonic (SH) coefficients so color can vary slightly with viewing angle. The order-0 (“DC”) coefficient is the base RGB color, which is all we need for clustering.

To render, every Gaussian is projected onto the screen as a soft ellipse (“splatted”), the ellipses are sorted by depth, and blended back-to-front. This renders photorealistically at real-time rates, which is why it replaced NeRF for interactive uses.

Two properties matter for us. First, splats are a *point-based* format, so grouping them into objects is a clustering problem, not a mesh-cutting problem. Second, moving a rigid group of Gaussians is easy: transform each mean by $\mu' = \mathbf{R}\mu + \mathbf{t}$ and rotate each covariance by $\Sigma' = \mathbf{R}\Sigma\mathbf{R}^\top$ (in practice, just multiply the stored quaternion). Nothing about the visual quality breaks when a splat cloud moves rigidly.

Where our splats come from. Feed-forward models (TripoSR, LGM) take one photo of an object and directly output a 3D representation in seconds, having learned shape priors from millions of 3D assets. They hallucinate the unseen back of the object, which is acceptable for simple convex shapes. The alternative is classical multi-view splatting from 30–60 photos, which is higher quality but more effort per scene.

4 Neural Networks and Graph Neural Networks

An MLP (multi-layer perceptron) is the basic block: alternating matrix multiplications and non-linearities that learn to map an input vector to an output vector. Every learnable piece of our model is an MLP; the architecture is about *how MLPs are wired together*.

Concept: Message passing on a graph

A graph has nodes and edges. A **graph neural network** (GNN) computes by repeated rounds of:

1. every edge computes a **message** from the feature vectors of its two endpoint nodes (an MLP does this);
2. every node **aggregates** the messages arriving on its edges (we sum them);
3. every node **updates** its own feature vector from the aggregate (another MLP).

One round lets information travel one hop. Ten rounds let it travel ten hops.

Why this matters

Physics *is* message passing. A contact force is literally a message between two touching particles: “I am here, moving this way, push back accordingly.” A GNN whose edges connect nearby particles has the same structure as the physical interaction, which is why this architecture family (a *relational inductive bias*) learns simulation well while a plain MLP on flattened coordinates learns nothing usable.

Three practical facts you will use constantly:

- **Permutation invariance:** summing messages means node ordering is irrelevant, so the same network handles any number of particles and objects.
- **Relative features only:** we feed edges the *difference* of endpoint positions, never absolute positions. The physics of a stack is the same anywhere on the table, and building that in (translation invariance) saves the network from having to learn it.
- **Normalization:** neural networks train well when inputs and targets have roughly zero mean and unit variance. Raw physics data does not (gravity dominates; contact spikes are rare and huge). We standardize every feature dimension using dataset statistics, and this single detail is the difference between a model that converges and one that does not.

5 Learned Simulation: The GNS Recipe

The architecture we build is a **Graph Network Simulator** (GNS, Sanchez-Gonzalez et al., ICML 2020), adapted for rigid bodies (in the spirit of FIGNet, ICLR 2023). The recipe:

1. **Encode:** lift each node’s raw features (recent velocities, mass, etc.) and each edge’s raw features (relative displacement) into 128-dimensional vectors with small MLPs.
2. **Process:** 10 rounds of message passing with residual connections.
3. **Decode:** read out an acceleration.
4. **Integrate:** apply the acceleration with semi-implicit Euler to get the next state. Repeat forever. Running the model repeatedly on its own output is called a **rollout**.

Concept: The two ideas that make rollouts stable

Idea 1: rigidity by construction. If the network predicts a separate acceleration for every particle, small per-particle errors accumulate and a solid block slowly deforms and melts over hundreds of steps. Instead we pool the processed particle features *within each body* and decode one linear plus one angular acceleration for the whole body (6 numbers). Every particle then moves under one rigid transform, so melting is impossible.

Idea 2: noise injection. A model trained only on perfect simulator states has never seen its own slightly-wrong predictions, so at rollout time errors compound and the simulation drifts. The fix: during training, corrupt the input state with small Gaussian noise while keeping the target computed from the clean state. The model thereby learns to *pull perturbed states back toward the truth*, which is exactly the skill a rollout needs. This is the single highest-leverage trick in the entire project.

Residual dynamics. We add gravity analytically in the integrator and let the network learn only the *rest* (contact and friction). Learning a residual around a known baseline is easier than learning everything, and it guarantees free fall is exactly right.

6 The Tool Stack

- **Python + uv:** uv is a fast package/environment manager. All training-side code is Python.
- **PyTorch on MPS:** PyTorch's Apple-GPU backend (`torch.device("mps")`). Caveats: no float64 (use float32 everywhere), occasional missing ops (set `PYTORCH_ENABLE_MPS_FALLBACK=1` so they fall back to CPU instead of crashing).
- **PyBullet:** a classical rigid-body physics engine with a simple Python API. Our ground-truth data factory. Has a GUI mode for eyeballing scenes.
- **trimesh:** mesh utilities: create primitive meshes, sample surfaces, convex hulls, inertia tensors.
- **Rerun (rerun-sdk):** a 3D visualization viewer you log data to from Python (`rr.log(...)`). You will use it dozens of times a day; it replaces writing any debug viewer.
- **h5py:** HDF5 files for the dataset (fast array storage with named groups).
- **HDBSCAN / RANSAC (scipy, scikit-learn, hdbscan):** clustering and robust plane fitting, used to split the reconstruction into objects and find the table surface. HDBSCAN finds clusters of varying density without being told how many clusters exist. RANSAC fits a model (here, a plane) by repeatedly fitting to random minimal subsets and keeping the fit with the most inliers, which makes it immune to outliers.
- **ONNX + ONNX Runtime Web:** ONNX is a portable saved-model format; ONNX Runtime Web executes it in the browser on WebGPU or WASM. This is how the trained network runs client-side.
- **Next.js + React Three Fiber (R3F):** the web app and its 3D canvas. R3F is React bindings for Three.js. A splat-rendering library (GaussianSplats3D or Spark) draws the Gaussians.

- **TripoSR / LGM:** pretrained single-image-to-3D models from Hugging Face. Downloaded once, never trained.

Part II

The Build

7 Phase 0: Environment and Skeleton

Step 1: Install the toolchain

Install uv (`curl -Lsf https://astral.sh/uv/install.sh | sh`), Node.js 20+ (for Phase 7), and the Rerun viewer. Verify `git`, `python3 --version (3.11+)`.

Step 2: Create the project and Python environment

```
uv init physsplat && cd physsplat
uv add torch numpy scipy pybullet h5py rerun-sdk hdbscan scikit-learn trimesh onnx
onnxruntime matplotlib
```

Step 3: Verify the Apple GPU works

```
uv run python -c "import torch; print(torch.backends.mps.is_available())"
```

Checkpoint: you are done with this step when

It prints `True`. If not, reinstall PyTorch; nothing else in Phase 2+ will be fast until this passes.

Step 4: Lay out the repository

```
physsplat/
  physsplat/
    datagen/      # Phase 1: scenes, sampling, dataset writer
    model/        # Phase 2: GNN, integrator, losses
    train/        # Phase 3: training loop, config
    eval/         # Phase 4: rollout metrics, video export
    recon/        # Phase 6: photo -> splats -> bodies -> particles
    export/       # Phase 7: ONNX export + parity tests
    common/       # shared: particle sampler, geometry helpers, constants
  web/           # Phase 7: Next.js app
  scripts/       # runnable entry points
  data/          # generated datasets (gitignored)
  checkpoints/   # trained models (gitignored)
```

Put shared constants in `common/constants.py` now: `DT = 1/60`, `GRAVITY = 9.81`, `PARTICLE_SPACING = 0.004`, `CONTACT_RADIUS = 1.5 * PARTICLE_SPACING`, `HISTORY = 5`. Many bugs later are two files disagreeing on one of these; a single source of truth prevents them.

Step 5: First Rerun hello-world

```
uv run python -c "
import rerun as rr, numpy as np
rr.init('hello', spawn=True)
```

```
rr.log('points', rr.Points3D(np.random.rand(100,3), radii=0.01))"
```

Checkpoint: you are done with this step when

The Rerun viewer opens showing 100 random points you can orbit. You now have a 3D debugger for the whole project.

8 Phase 1: The Synthetic Dataset

Why this matters

Everything the neural network will ever know about physics comes from this phase. The model never sees photos or Gaussians during training; it sees only what you generate here: surface-point clouds of simple objects moving under PyBullet's physics. Quality bar: if a recorded trajectory looks wrong to your eye in Rerun, it will poison training.

Step 6: Define the object family

In `datagen/shapes.py`, write a function `random_shape(rng)` returning a description (type, dimensions) drawn from: box (edges 2–8 cm, aspect up to 3:1), cylinder/capsule (radius 0.3–1.5 cm, length up to 16 cm, covering pencils), random convex polyhedron (convex hull of 10–20 random points, scaled to 2–8 cm). Also write `shape_to_trimesh(shape)` producing a `trimesh` mesh, and compute mass and the inertia diagonal from the mesh at fixed density (trimesh provides `mesh.mass_properties`).

Step 7: Build one static scene in PyBullet

In `datagen/scenes.py`: connect with `p.connect(p.GUI)`, add a ground plane, spawn 2–6 random shapes. Implement three placement regimes and pick one at random per scene: (a) a rough stack with small horizontal offsets, (b) objects scattered lying flat (the pencil-on-desk case), (c) a drop from 5–15 cm. Randomize friction $\mu \in [0.3, 0.9]$ and restitution $\in [0, 0.4]$ per object.

Checkpoint: you are done with this step when

Running the script opens the PyBullet window and you watch plausible scenes settle. Watch at least ten. Anything jittering, exploding, or sinking into the floor gets fixed now.

Step 8: Record trajectories

Simulate at 240 Hz internally (PyBullet default is fine) and record every 4th step, giving 60 Hz data. After a 100-step settle period, record 300 frames of: per-body position, quaternion, linear and angular velocity (`p.getBasePositionAndOrientation`, `p.getBaseVelocity`).

Step 9: Add recorded forces (the poke-and-grab channel)

Two action kinds per trajectory, sharing one encoding. *Pokes*: at 1–3 random frames, a short force burst at a random surface point of a random body (calibrate magnitudes so pokes visibly move

objects without launching them off-screen). *Grabs*: in about half of trajectories, attach a spring-damper from a random surface point (in pile scenes, often on a load-bearing body) to a target that moves along a smooth random path for 0.5–2s: lift, sideways pull, or slide, then release. Apply both via `p.applyExternalForce`; save frame index, body id, application point, and force vector per step. This is the action channel: at inference a mouse poke or grab is encoded identically, and the grab trajectories are what teach the model that pulling a supporting pencil out of a pile makes the rest collapse.

Common trap

PyBullet’s `applyExternalForce` must be re-applied every internal substep for the duration you intend, and uses flags `p.WORLD_FRAME`. Forgetting the re-apply gives impulses $4\times$ weaker than logged, and the model later learns wrong poke responses that you will chase for days in Phase 5.

Step 10: Write the shared particle sampler

In `common/particles.py`, the most reused function in the project:

```
def sample_surface(mesh: trimesh.Trimesh, spacing=PARTICLE_SPACING, cap=400):
    # oversample the surface, then farthest-point-subsample to ~uniform spacing
    dense, _ = trimesh.sample.sample_surface(mesh, 20000)
    picked = [0]
    d = np.linalg.norm(dense - dense[0], axis=1)
    while d.max() > spacing and len(picked) < cap:
        i = int(d.argmax()); picked.append(i)
        d = np.minimum(d, np.linalg.norm(dense - dense[i], axis=1))
    return dense[picked]          # (P, 3) in the mesh's own (body) frame
```

Points are returned in the *body frame*; store them once per object as offsets $\mathbf{r}_i$ (relative to the center of mass). World positions at any frame are $\mathbf{x}_i = \mathbf{R}_t \mathbf{r}_i + \mathbf{t}_t$.

Why this matters

Fixed *spacing* (not fixed count) means a pencil and a big block have identical local point density, so one global contact radius works for all shapes. And because Phase 6 will run *this same function* on reconstructed objects, training data and real data become statistically interchangeable, which is the entire reason sim-to-real transfer works here.

Step 11: Write the dataset to HDF5

In `datagen/writer.py`: one HDF5 group per trajectory containing per-body constant data (offsets $\mathbf{r}$, mass, inertia diagonal, friction, restitution) and per-frame data (position, quaternion, velocities, impulse channel). Store poses, not world-space particle clouds: it is $100\times$ smaller and guarantees the ground truth is exactly rigid.

Step 12: Visual audit in Rerun

Write `scripts/view_traj.py`: load a trajectory, reconstruct world-space particles per frame, log to Rerun with per-body colors, log impulse arrows when active.

Checkpoint: you are done with this step when

Scrubbing the Rerun timeline shows: stacks settle and stay still; pokes visibly shove objects; cylinders roll and stop; nothing interpenetrates the floor. Audit at least 10 random trajectories.

Step 13: Generate the full dataset

Parallelize with `multiprocessing` (one PyBullet DIRECT-mode instance per worker), seed per trajectory for reproducibility. Target 5,000 trajectories. Split 90/5/5 into train/val/test *by trajectory*, never by frame (frames of one trajectory are nearly identical; splitting by frame leaks test data into training).

Step 14: Compute normalization statistics

Over the training split only, compute mean and standard deviation of: velocity features, acceleration targets (linear and angular, per-body), and edge displacement features. Save to `data/stats.json`. These exact numbers ship all the way to the browser in Phase 7.

Step 15: Build the PyTorch Dataset

In `train/dataset.py`, a `__getitem__` that picks a trajectory and frame $t \geq 5$ and returns: world particle positions at t , the 5-step velocity history (finite differences of positions), body ids, per-body mass/inertia, the impulse channel at t , and the *target*: each body's actual linear and angular acceleration from t to $t+1$ minus gravity (the residual the network must learn). Apply the domain-randomization corruptions here with a flag: 0.5–2 mm surface noise on offsets, random 5–15% particle dropout.

Checkpoint: you are done with this step when

A quick script draws 100 samples, checks shapes and dtypes (all float32), checks targets are finite, and histograms the normalized targets: roughly zero-mean, unit-variance, heavy-tailed (contact spikes). If the histogram is not centered, your gravity subtraction or normalization is wrong.

9 Phase 2: The Graph Network Simulator

Step 16: The graph builder

In `model/graph.py`: given particle positions, return the edge list $\{(i, j) : \|\mathbf{x}_i - \mathbf{x}_j\| < r_{\text{contact}}\}$. Implement with a uniform grid hash: bucket points into cells of size r_{contact} , compare only within neighboring cells. Also return per-edge features: displacement $\mathbf{x}_j - \mathbf{x}_i$, its norm, and a same-body flag.

Step 17: Unit-test the graph builder

Compare against the brute-force $O(N^2)$ distance matrix on 20 random point sets. Exact match required.

Checkpoint: you are done with this step when

The test passes, and building the graph for a 1,500-particle scene takes under a couple of milliseconds. This function is called every physics step forever after; it must be both correct and fast.

Step 18: Encoder and processor

In `model/gnn.py`, plain PyTorch, no PyTorch Geometric:

```
class MPBlock(nn.Module):
    def forward(self, h, e, senders, receivers):
        msg_in = torch.cat([e, h[senders], h[receivers]], dim=-1)
        e = e + self.edge_mlp(msg_in) # update edges
        agg = torch.zeros_like(h).index_add_(0, receivers, e) # sum messages
        h = h + self.node_mlp(torch.cat([h, agg], dim=-1)) # update nodes
        return h, e
```

Encoder: 2-layer MLPs lifting node features (15–20 dims) and edge features (5 dims) to width 128 with LayerNorm. Processor: 10 MPBlocks.

Common trap

Get `senders/receivers` orientation consistent everywhere, and include both edge directions (i, j) and (j, i) . A silent asymmetry here trains to mediocre loss and terrible rollouts, and is miserable to find later. Write a 5-particle hand-checkable test now.

Step 19: The per-body rigid decoder

Mean-pool final node embeddings within each body (an `index_add_` divided by counts), then a 2-layer MLP to 6 outputs: $\Delta \mathbf{v}_b$ and $\Delta \boldsymbol{\omega}_b$, in normalized units.

Step 20: The fallback per-particle decoder plus Kabsch projection

Also implement: per-node MLP to 3D accelerations, free integration, then snap each body to the best-fit rigid transform. Kabsch: with predicted positions $\tilde{\mathbf{x}}_i$ and offsets $\mathbf{r}_i$, form $\mathbf{A}_b = \sum_i (\tilde{\mathbf{x}}_i - \bar{\mathbf{x}}_b) \mathbf{r}_i^\top$, SVD $\mathbf{A}_b = \mathbf{U} \mathbf{S} \mathbf{V}^\top$, then $\mathbf{R}_b = \mathbf{U} \text{diag}(1, 1, \det(\mathbf{U} \mathbf{V}^\top)) \mathbf{V}^\top$ and $\mathbf{t}_b = \bar{\mathbf{x}}_b$. Both heads share the encoder/processor; select by config flag.

Why this matters

The per-body head is the primary design (rigidity is exact, output is tiny). But pooling can blur *where* on a body contact happened; if it underfits in Phase 3, you switch one flag rather than redesigning. Cheap insurance built at the right time.

Step 21: The integrator

In `model/integrator.py`: denormalize predictions, add gravity, semi-implicit Euler per body, rotation update via rotation-vector exponential, then particles from body poses. Keep it a pure function of (state, prediction) $\rightarrow$ state; Phase 7 reimplements it in TypeScript and purity makes the two easy to compare.

Step 22: Assemble the one-step model and overfit a single trajectory

Wire encode $\rightarrow$ process $\rightarrow$ decode into one module. Then the most diagnostic experiment of the project: train on *one* trajectory until the loss approaches zero.

Checkpoint: you are done with this step when

Single-trajectory loss falls by 3–4 orders of magnitude and its rollout visually replays the trajectory. If it will not overfit, the bug is in features, normalization, or indexing, and no quantity of data will hide it. Do not proceed until this passes.

Step 23: The rollout function

`rollout(model, initial_state, actions, n_steps)`: loop of graph-build, predict, integrate, maintaining the 5-step velocity history. Log any rollout to Rerun with one call. You will run this function thousands of times.

10 Phase 3: Training

Step 24: The training loop

`train/loop.py`: AdamW, learning rate 10^{-4} decayed exponentially to 10^{-6} ; batches of 2–4 scenes collated into one big disjoint graph (concatenate nodes, offset edge indices); MSE on normalized 6D body accelerations; gradient clip at 1.0; checkpoints and a small metrics log (loss curves, learning rate) every few hundred steps. Build graphs in dataloader workers, not the training step.

Step 25: Noise injection

Implement the random-walk corruption from Section 5: per sample, add accumulated Gaussian noise (total $\sigma \approx 1\text{--}3\text{ mm}$) to the position/velocity history while the target still points to the true clean next state. Make σ a config value; you will tune it.

Why this matters

Without this, your Phase 4 rollouts will drift within a second or two and you might wrongly conclude the whole approach fails. With it, rollouts of many hundreds of steps stay plausible. It is the difference between the project working and not.

Step 26: Short training run on MPS

Train 20k steps on 10% of the data. Watch: loss decreasing smoothly, throughput (steps/sec), MPS actually in use (Activity Monitor GPU tab). Fix input-pipeline stalls now.

Step 27: Rollout-based validation

Every 2,500 training steps, run 5 held-out validation rollouts of 300 steps and log translation/rotation error vs. ground truth. Single-step loss is a poor proxy for rollout quality; this metric is your real compass.

Step 28: Full training run

300k–500k steps on the full dataset. On the M4 Pro expect 1–3 days wall clock; alternatively rent one A100 day (< \$30), the code is device-agnostic. While it runs, start Phase 5’s scaffolding.

Step 29: Qualitative milestones, in order

Check these against rollouts as training progresses: (1) a dropped box falls, lands, stops; (2) a two-box stack stays standing indefinitely; (3) a poked stack topples plausibly; (4) a flicked pencil rolls and comes to rest. They tend to emerge in roughly that order; (4) is the hardest.

Step 30: Rollout fine-tuning

From the converged checkpoint: 20k further steps on 4–8-step unrolled loss (backprop through the integrator, which is differentiable) at learning rate 10^{-5} . Tightens long-horizon behavior.

Step 31: Head comparison (only if needed)

If milestone (3) or (4) fails after tuning noise σ and rerunning: flip the config flag to the per-particle + Kabsch head and retrain. Keep whichever wins on validation rollout error.

Checkpoint: you are done with this step when

Phase 3 exit: all four qualitative milestones pass, and 300-step validation rollout error is stable run-to-run.

11 Phase 4: Evaluation

Step 32: Metrics harness

`eval/metrics.py` over the held-out test split: per-body translation and rotation error at 50/150/300 steps; interpenetration depth (bodies and floor); stability (fraction of initially-at-rest scenes still at rest after 300 unperturbed steps); energy trend over passive rollouts (must not grow).

Step 33: Side-by-side videos

Same initial state and same pokes through PyBullet and through your model, rendered side by side (Rerun screen capture or matplotlib animation). These become the centerpiece of the README.

Step 34: Stress tests

Taller stacks than training ever saw, more objects, maximum pokes, extreme aspect ratios. Note where it breaks; these bounds become the demo’s guardrails (e.g., cap user impulse strength at the training maximum).

Step 35: Numbers table

Freeze one table of the metrics for the writeup, tied to a named checkpoint file. This checkpoint is now “the model” for all later phases.

Step 36: Failure gallery

Save 3–5 characteristic failures (a pencil that jitters at rest, a stack that leans). Honest limitations make the writeup stronger, and you will recognize regressions instantly later.

12 Phase 5: Local Interactive Demo

Why this matters

Before any reconstruction or browser complexity, prove the *interaction loop*: click, poke, watch physics respond, reset. This phase uses synthetic scenes rendered as plain meshes, a Python server running the model on MPS, and a minimal web page. Every hard problem solved here (impulse encoding, reset, pacing) transfers directly to Phase 7.

Step 37: The scene packet format

Define the JSON+binary packet that every later phase reuses: per-body mesh (for now) or Gaussian data (later), particle offsets, masses, inertia diagonals, initial poses, normalization stats, and constants (Δt , contact radius). Version it.

Step 38: WebSocket physics server

FastAPI + WebSocket: on connect, send the packet; then receive `{impulse: body, point, vector}` messages and stream back per-body poses at 60 Hz from a continuous model rollout on MPS.

Step 39: Minimal Three.js client

One HTML page: render each body as a colored mesh, orbit controls, update poses from the socket.

Step 40: Mouse-to-force: pokes and grabs

On mouse down: raycast into the scene, hit-test body bounding boxes. A quick drag-release becomes a poke, a short force burst in the drag direction. Press-and-hold becomes a grab: a spring-damper force from the grab point toward the mouse target, recomputed every step until release, using the same spring gains as data generation. The server encodes both into the action feature $\mathbf{a}^{\text{ext}}$ on the hit body's particles with Gaussian falloff from the hit point, exactly matching the training-data encoding. Cap force magnitudes and target speeds at the training distribution's maxima.

Common trap

If pokes in the demo feel wrong but Phase 4 videos looked right, the impulse *encoding* mismatch between datagen and server is the culprit almost every time. Unit-test that both code paths produce identical action features for the same nominal poke.

Step 41: Reset and settle behavior

Reset button restores the initial packet state. Add the demo-hygiene damping: when a body's kinetic energy stays below a small threshold for 30 steps, gently zero its velocities to suppress micro-jitter at rest.

Step 42: Playtest

Ten minutes of ruthless play. Note everything that feels off; fix what is cheap now.

Checkpoint: you are done with this step when

Phase 5 exit: you can knock over a synthetic stack with your mouse on your own machine, it feels responsive, reset works, and objects at rest stay visually still.

13 Phase 6: The Reconstruction Front End

Step 43: Capture test photos

3–5 *distinctly colored* objects (start with blocks and erasers; add a pencil later), plain matte background, soft even light, slightly elevated camera angle. Take 10 candidate photos.

Step 44: Background removal

`rembg` on each photo to isolate the object arrangement.

Step 45: Run single-image reconstruction

Start with TripoSR (Hugging Face checkpoint; runs on MPS, if an op fails use the fallback env var or free Colab). It outputs a triplane/mesh; extract the mesh, sample its surface densely with color, and instantiate one small isotropic Gaussian per sample. If quality disappoints, try LGM (native Gaussian output, heavier). Save as `.ply`.

Checkpoint: you are done with this step when

Loaded into Rerun as colored points, the reconstruction is recognizably your objects from all sides, hallucinated back included. Judge shape and color separation, not beauty.

Step 46: Gravity alignment and metric scale

`recon/canonicalize.py`: rotate so the support plane normal is $+z$ (RANSAC, next step, can be run first to find it); scale so the largest object dimension is plausible in metric units for its class (assumed, not estimated).

Step 47: Ground plane via RANSAC

Fit a plane to the lowest 30% of points by z ; label inliers as static ground. If the reconstruction has no ground (common for single-image), insert a synthetic plane at minimum z .

Step 48: Cluster into objects

HDBSCAN on $[\mu/\sigma_{xyz} \parallel \lambda \text{rgb}]$ (position normalized, color weighted). Tune λ and `min_cluster_size` on your test scene; merge or drop tiny clusters. Log the result to Rerun colored by cluster id.

Checkpoint: you are done with this step when

Each real object is exactly one cluster, the ground is separate, and floaters are gone. This checkpoint is pure parameter tuning; budget an afternoon.

Step 49: Per-body physical properties

Per cluster: convex hull (trimesh), center of mass, oriented bounding box (for picking), mass and inertia diagonal from the hull at assumed density.

Step 50: Resample physics particles

Run `common/particles.sample_surface` (the *identical* Phase 1 function) on each cluster's hull surface. Store body-frame offsets. Log to Rerun next to a training-data scene: the two particle clouds should be statistically indistinguishable in spacing and coverage.

Step 51: Emit a scene packet

Wire it all into the Phase 5 packet format (meshes from hulls for now).

Step 52: End-to-end local test

Load the photo-derived packet into the Phase 5 demo. Poke your reconstructed objects.

Checkpoint: you are done with this step when

Phase 6 exit: a photo from your phone becomes an interactive physics scene on your machine. Expect one round of debugging; scale or gravity-axis mismatches are the usual suspects, and the domain randomization from Step 16 is what absorbs reconstruction lumpiness.

14 Phase 7: Browser Inference and Splat Rendering

Step 53: Export the model to ONNX

`export/to_onnx.py`: export with dynamic (symbolic) node and edge dimensions. The graph builder and integrator are *not* exported; only encode-process-decode is. If the fallback head shipped, note its SVD does not export; the Kabsch step will be TypeScript.

Step 54: Parity test, Python vs. ONNX Runtime

Drive both the PyTorch model and the ONNX session with identical inputs over a 100-step rollout (graph building and integration in shared Python).

Checkpoint: you are done with this step when

Positions agree within 10^{-4} over the full rollout. Do not write a line of frontend model code before this passes; debugging numerics through a browser is misery.

Step 55: Scaffold the web app

`npx create-next-app web` with TypeScript; add React Three Fiber, drei, `onnxruntime-web`, and a splat renderer (GaussianSplats3D or Spark).

Step 56: TypeScript graph builder and integrator

Port the grid-hash neighbor search and the semi-implicit $SE(3)$ integrator. Both are pure functions in Python by design; port them line by line, then run the cross-language parity test: same packet, 100 steps, TS+ONNX vs. Python, agreement to 10^{-3} .

Step 57: The physics loop in the browser

A `requestAnimationFrame`-decoupled loop: build graph, run the ONNX session (WebGPU provider, WASM-SIMD fallback), integrate, at a fixed 60 Hz accumulator (or 30 Hz with render interpolation). Measure per-step time in the console.

Checkpoint: you are done with this step when

A reconstructed scene (about 1,000 particles) steps in single-digit milliseconds and the loop holds rate with the tab in the foreground.

Step 58: Splat rendering with per-body transforms

Load the scene's Gaussians grouped by body id; give each body its own splat batch with a model matrix. Per frame, write each body's pose into its matrix: means transform by $(\mathbf{R}_b, \mathbf{t}_b)$; covariances rotate by quaternion multiplication. The library handles depth-sort refresh.

Why this matters

Per-body batches mean the CPU uploads 6 numbers per body per frame instead of rewriting a 100k-Gaussian buffer. This is what makes 60 FPS trivial instead of impossible.

Step 59: Port interaction

OBB raycast picking, drag-to-impulse (same encoding as Phase 5, now computed client-side), gravity and impulse-strength sliders, reset.

Step 60: The scene endpoint, or static packets

Two options: (a) FastAPI on Modal running Phase 6 for user-uploaded photos; (b) precompute packets for several scenes and serve them statically. Ship (b) first: zero cost, zero cold starts, reviewers still get full interactivity on real photo-derived scenes. Add (a) later if desired.

Step 61: Deploy

Vercel Hobby tier. Check the ONNX WASM/WebGPU assets ship correctly (they need specific headers for threading; Vercel config handles it).

Step 62: Cross-device pass

Test Chrome, Safari, Firefox, and a phone. Safari lacks WebGPU in places; confirm the WASM fallback engages and remains fast enough at 30 Hz.

Checkpoint: you are done with this step when

Phase 7 exit: a public URL where anyone can orbit and poke your photo-derived objects, with all physics running in their browser.

15 Phase 8: Polish and Writeup

Step 63: UI polish

Dark theme, a short “what am I looking at” overlay explaining that the physics is a learned neural network running locally in the visitor’s browser, scene selector, FPS counter.

Step 64: The demo video

60–90 seconds, screen-recorded: the source photo, the reconstruction, then interaction. This is what most people will watch instead of clicking.

Step 65: README and writeup

Architecture diagram, the Phase 4 metrics table, side-by-side videos, honest limitations (from the failure gallery), and pointers to the three ideas that made it work: rigidity by construction, noise injection, and the shared particle sampler. Those three are the story.

Checkpoint: you are done with this step when

Someone who has never spoken to you can open the URL, understand what they are seeing within 30 seconds, knock over your blocks, and find the code and writeup from the page. That is the finish line.

Glossary

3DGS / splat Scene format made of colored transparent 3D Gaussians; renders photorealistically in real time.

Body frame Coordinate system attached to an object; particle offsets live here.

GNS Graph Network Simulator; the encode-process-decode GNN recipe for learned physics.

HDBSCAN Density-based clustering that finds a variable number of clusters; used to split the splat cloud into objects.

Kabsch SVD-based algorithm giving the best-fit rotation between two point sets; used to snap predictions back to rigidity in the fallback head.

MPS Metal Performance Shaders; PyTorch’s Apple-GPU backend.

Noise injection Training-time corruption of inputs that teaches the model to correct its own rollout errors.

ONNX Portable neural-network file format; ONNX Runtime Web executes it in the browser.

Packet This project’s serialized scene bundle: Gaussians, particles, body metadata, constants.

RANSAC Robust fitting by consensus of random minimal subsets; used for the ground plane.

Residual dynamics Learning only what remains after known physics (gravity) is handled analytically.

Rollout Running the one-step model repeatedly on its own predictions.

Semi-implicit Euler Integration scheme: update velocity first, then position with the new velocity.

SE(3) The space of rigid poses (rotation + translation).